

# Observability

**A hands-on course in building agents by building their harness**

Lesson 7

# The terminal scrolled past, and the answers went with it

Lessons 5 and 6 made the harness safe to leave running. Now you run it where you can't watch it:

- Why did that take four minutes?
- Why did it cost that much?
- What did it run, and in what order?
- Which command did the guard refuse, and which did the box kill?

# A record of what the harness did, kept as it does it

- Every call to the model
- Every tool it ran
- How long each took, and how many tokens each cost
- Every time a guardrail said no

A harness you can't see into is one you can't trust or fix.

# Same run, two records, two readers

|             | Episode (Lesson 4)                  | Trace (Lesson 7)                            |
|-------------|-------------------------------------|---------------------------------------------|
| Written for | the model                           | you, the person running quark               |
| Holds       | what working memory would have been | timings, token counts, exit codes, refusals |
| Used to     | search its own past                 | answer your questions afterwards            |

A record for the model is context. A record for the operator is observability.

## Built on control flow, and only control flow

- The model interface only knows about one call
- Output only knows about one tool
- The loop is where a call, its tool, the guard's decision and the next call all happen, in order
- So the loop is the one place a record of the run can be made

No new primitive: a production layer adds hardening.

# Read off what's already there, and write it down

- **A trace.** One JSON line per thing that happened, appended in order
- **Timing.** Read the clock before and after a model call or a tool
- **Token accounting.** Every response says tokens in, out, and read from or written to the cache
- **Cost** is tokens times your provider's price; you keep the rates yourself

Observability only watches. It never changes what's sent, what runs, or when the loop stops.

# `trace()`: one line per step, for whoever runs quark

Lesson 6's `quark.py` plus 12 lines. `time` joins the imports, and at the top of `# — control flow —` (abridged):

```
def trace(**event):  
    os.makedirs(".quark", exist_ok=True)  
    with open(".quark/traces.jsonl", "a") as f: f.write(json.dumps({...}) + "\n")
```

- Each line is `ts`, the `episode` this run is written to, and whatever you pass it
- Append-only: a run that crashes halfway still leaves everything before the crash
- `episode` ties the two records together

# Then one call wherever something happens

Each is a single line next to code that was already there:

```
trace(event="start", input=input)
    trace(event="stopped", steps=steps, tokens=spent)
    trace(event="too_long", drop=drop)
        trace(event="refused", cmd=block.input["cmd"], why=no)
```

- **start:** where a run begins and what came in
- **stopped:** the steps and tokens a limit stopped at
- **too\_long:** the API said the prompt was too long, so compaction runs
- **refused:** the command the guard wouldn't run, and why

# The model call and the tool are timed

```
start = time.time()
response = call(max_tokens=16384, system=system(), tools=tools, messages=working_memory)
trace(event="model", seconds=round(time.time() - start, 2), ...)
    start = time.time()
    done = subprocess.run(..., ...)
    trace(event="tool", ..., seconds=round(time.time() - start, 2), exit=done.returncode, ...)
```

(abridged)

- **model** also records `stop_reason` and four counts from `response.usage`:  
input, output, cache read, cache write
- **tool** also records the command and how many characters it printed
- A failed command has a non-zero `exit`; one the box killed is `137`

# Run it: nobody there to answer the guard

```
quark.py "which file in this folder has the most lines? answer in one sentence" < /dev/null
```

```
$ wc -l * 2>/dev/null | sort -rn | head -5
allow `wc -l * 2>/dev/null | sort -rn | head -5`? [y/N] [the person said no]
$ wc -l * | sort -rn | head -5
allow `wc -l * | sort -rn | head -5`? [y/N] [the person said no]
$ wc -l *
5000 big.txt
  2 small.txt
5002 total

`big.txt` has the most lines, at 5,000.
```

# What you see afterwards, in `.quark/traces.jsonl`

| event                     | seconds | stop_reason | in  | out | cache_read | cache_write |
|---------------------------|---------|-------------|-----|-----|------------|-------------|
| model                     | 2.42    | tool_use    | 95  | 70  | 0          | 6609        |
| model                     | 1.77    | tool_use    | 180 | 86  | 6609       | 0           |
| model                     | 1.58    | tool_use    | 281 | 81  | 6609       | 0           |
| tool <code>wc -l *</code> | 0.11    | exit 0      |     |     |            |             |
| model                     | 1.0     | end_turn    | 390 | 20  | 6609       | 0           |

The trace's fields as a table; the `start` line and two `refused` lines are left out.

# One JSON object per line, so you can ask it questions

Add up one run's model calls, by its episode, with `jq`:

```
{"calls":4,"input":946,"output":257,"cache_read":19827,"cache_write":6609,"seconds":6.77}
```

The whole run: four calls, 6.77 seconds, and most of what it read came from the cache.

# A run where things go wrong

A copy with the sandbox's `TIMEOUT` lowered to 5, and `y` piped in for the guard (shortened):

```
$ ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory

$ sleep 60
allow `sleep 60`? [y/N]
(killed: ran over 5 seconds or out of memory)
```

The agent told you what happened, this time. Without the trace, you'd only know if you were watching.

# Everything that didn't go to plan, across both runs

```
jq -c 'select((.event=="tool" and .exit!=0) or .event=="refused")' .quark/traces.jsonl
```

| event   | cmd                                      | seconds | exit / why         |
|---------|------------------------------------------|---------|--------------------|
| refused | wc -l * 2>/dev/null   sort -rn   head -5 |         | the person said no |
| refused | wc -l *   sort -rn   head -5             |         | the person said no |
| tool    | ls /nonexistent                          | 0.12    | 2                  |
| tool    | sleep 60                                 | 5.11    | 137                |

# What else observability can be

- **What's recorded:** events, *spans* with a parent (a tree), or every full request and response
- **Where it goes:** a file, the terminal, a database, an OpenTelemetry collector
- **When you see it:** afterwards, a live line per step, or a dashboard that alerts you
- **What's worked out:** tokens, cost, latency, cache hit rate, tool failures, steps per task
- **How you ask and how long it's kept:** `grep` and `jq`, SQL or a trace UI; sampling, rotation, deletion

# A product on its own, and a fuller example

Tracing platforms like Langfuse and LangSmith are this layer: you send them spans, they store them, price them and show them as a tree.

`observability.py` , built on Lesson 3's `control_flow.py` , does more of that:

- **Spans:** `span()` records an id, a `parent` , a `start` and `seconds` ; a crash is recorded and still crashes
- **Cost:** `cost()` multiplies the four token counts by `PRICE` (example rates)
- **A live line** per step, **a summary** per run, and **a report** across every run

# One run says what happened; every run says what's normal

`observability.py "run ls /nonexistent, then run sleep 3 ..."` prints a line per step (shortened):

```
[step 1: model 1.96s, 397 tokens in, 166 out, $0.0037]
$ ls /nonexistent
[step 1: tool 0.0s, exit 2]
$ sleep 3
[step 1: tool 3.0s, exit 0]
[step 2: model 1.35s, 645 tokens in, 71 out, $0.0030]
```

`observability.py report` , three runs (shortened):

| run      | calls | tools | prob. | time | cost     |
|----------|-------|-------|-------|------|----------|
| 7941924a | 2     | 1     | 0     | 2.2s | \$0.0045 |

## The rule

Record what the harness does as it does it: each model call, each tool, each refusal, how long it took and what it cost, in a log you can search afterwards.

Do it where control flow already sees the whole sequence, read from what's already there, and keep it apart from what the model remembers.

# What observability never does

- **Control flow:** it sits in the loop, but never decides what runs next or when to stop
- **Model interface:** sends and receives exactly as before; it only reads `usage`
- **Input:** it gathers none
- **Context:** the trace is not in the request; that's what makes it observability
- **Output:** runs tools the way it always did, only with a clock around them

## What's missing: it watches, and that's all

It will faithfully record that:

- the API dropped a call halfway through a long run
- the model was cut off in the middle of a command
- the process died and took the work with it

It can tell you exactly where things broke; it can't pick up from there. A harness that runs unattended has to survive a bad day, not just describe one.

# **Next: Resilience**

Lesson 8